

interface b/w computer hardware and user application

Program → set of instructions written in a programming language that perform

a specific task or set of tasks. (Passive entity)
location → Disk

Process → a program that is currently running
(Active entity)
location → Ram

Thread → smallest unit of execution inside a process.
o Process can have multiple threads.

o Thread share the same memory.

(ACTIVE entity)

location → Process.

Passive → (Not running)
do not do anything on its own

Active entity
→ current task work
now running

Multiprocess - multiple process run at same time (Parallel)

Multithreading → Process divided into multiple threads
threads run concurrently
share same memory,

Multiprogramming → multiple programs are kept in memory at same time.

• CPU switch b/w them to keep them busy.

Multitasking → one CPU handles multiple tasks by switching very fast.

gives illusion of parallel execution

(single CPU)

processes

processes

processes

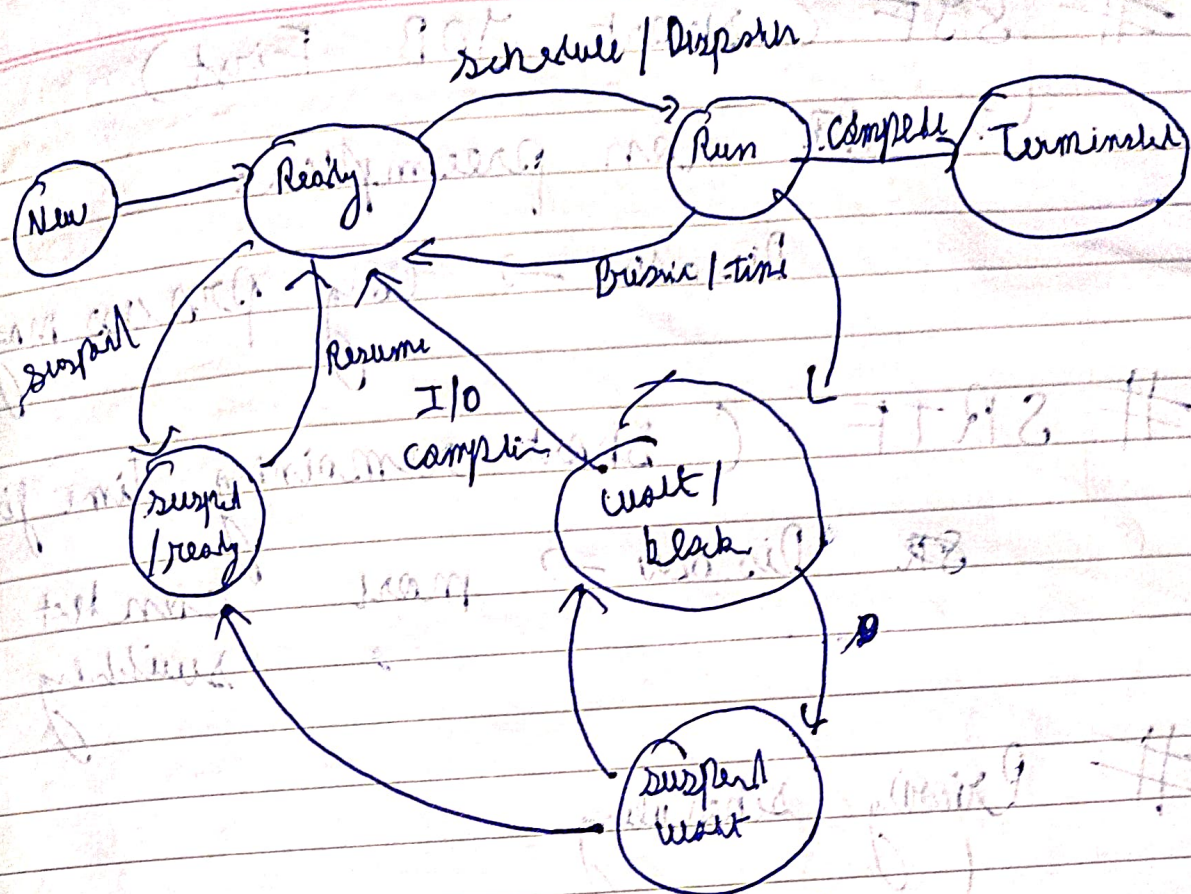
processes

processes

processes

processes

processes



CPU scheduling Algo

① FCFS — non preemprive — once process start running — it continues until complete

⊗ Issue ⊗

↳ ~~starvation~~ — Poor utilizsi of the CPU if long processes arrives before short JOB

° convoy effect —> long process — delay other

SJF (Short Job First)

↳ ~~non~~ non preemptions

Disadv → long process may starve

SRTF (Short remaining time first)

~~Dis~~ Disadv → more context switching

Priority scheduling

↳ Since priority high → ~~also~~ less delay

• Preemption - non batch

Dis Adv → starvation (low priority wait forever)

Sol → Aging → Priority inc ~~over time~~ overtime.

R.R. - Round Robin

o todos equal chance (time slice)

o Preemptive

o Time Quantum
↳ fixed time

Process runs for quantum
if not finished $\rightarrow$ goes to end of ~~queue~~ queue.

Multilevel queue scheduling

Process can move between queues

rules

- o high priority queue first get its execution
- o if process uses too much CPU $\rightarrow$ move down
- o Short job stay in high queue

It is complex

critical section is just a part of the program that work with shared things so we need rule (like a lock) to keep it safe so only thread can use it at a time.

Process Synchronization

make sure diff task or parts of a program happen in the right order so they don't mess up when they use shared resource.

How are the Key requirements of Synchrony Mechanisms.

① Mutual Exclusion $\rightarrow$ one process can ~~not~~ be in critical section at a time.

② Progress $\rightarrow$ must be done.

Bounded execution Time for go to critical section

Process Synchronization mechanism

1) Locks / mutex

2) Semaphores → count semaphores allow a specified number of processes or threads to access a shared resource simultaneously.

• Provide mechanism for mutual exclusion, signaling & coordination.

3) Read-Write lock → Read can anyone ~~read~~ but write can be one at a time

Deadlock $\rightarrow$ each of the computer processes waiting for some

resource that has been assigned to some process.

Challenge

So NO process gets executed since the resource it needs.

Conditions

(1) mutual ~~ex~~ exclusion $\rightarrow$

(2) Hold & all wait of resource

(3) NO preemption $\rightarrow$ NO end when complete full

(4) circular wait $\rightarrow$

Deadlock Handling Technique

- ① Deadlock Prevention
- ② Deadlock Avoidance
- ③ Deadlock Detection
- ④ Deadlock Recovery → remove process in deadlock
- ⑤ Deadlock Ignorance

Memory management

- ① fixed partitioning → internal fragmentation

- ② dynamic partitioning

↓
Algo

- ① FIRST - FIT

- ② Best fit

- ③ Worst fit

Paging - memory management technique

Physical and logical memory

are divided into fixed size blocks

{ Instead of giving program a big chunk of memory the OS divides memory into small pieces (pages)

Then keep track of which page goes where so each program is this it has big

continuous memory even though it all ~~breaks~~ is broken up.

Virtual memory

big imaginary extra space on top of your ram

if physical memory fills up the system temporary use a part of it as if it were ram.

various memory are in mix RAM + DISK

Page replacement algo

↳ To Keep which page in memory

① FIFO ~~Page~~ PRA

↳ Remove side page in memory

② CRU $\rightarrow$ (P. Least Recent used)

Remove the page Not used for long time.

③ optimal RA $\rightarrow$ Remove page that will not be in memory for long time in future.

④ LFU $\rightarrow$ Least frequently used

⑤

216 Thrashing

↳ so much swapping b/w Ram and disk so that it no longer doing useful work.

Segmenting → divide program

mems into diff part or segment like code, data, stack, each segment has its size & can grow and shrink independently.

helps organ memory

Disk management

↳ organised and control storage on hard drive like Keep Track of file

① FCFS

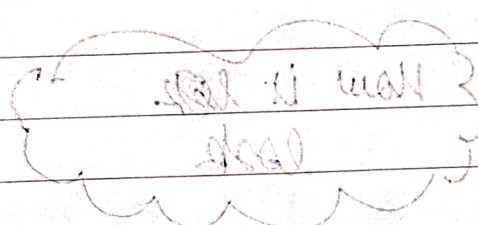
② ~~SS~~ Short Seek Time first

③ SCAN

④ C SCAN

⑤ LOOK

⑥ C LOOK



also called seek time → head is moving back and forth